SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

Week 12 — Question Paper (Comprehension-Based MCQ / MSQ)

Comprehension 1 — Seeding a Campaign: Cores and Going Viral

Case Study

A social-media growth team, led by Omar, wants a campaign clip to go viral. Anything that spreads across the internet — a picture, a video, a slogan — they loosely call an internet meme, and they know only a tiny fraction of clips ever catch on. Three levers shape whether a clip spreads: the quality of the clip itself, the structure of the platform's follower network (which is fixed and the same for everyone), and the choice of which accounts to seed it with. Since they cannot rewrite the network, their effort goes into picking the right accounts to infect.

Which accounts are the “right” ones? Candidates include the highest-degree account, the most central by closeness, or the best connector by betweenness. The team learns that the most reliable target is the network's core. Many real networks have a core–periphery shape: a small, densely interconnected core surrounded by a sparsely connected periphery. Seed one core account and, because the core is so tightly knit, the idea races through the whole core and from there out to the periphery.

To locate the core precisely the team uses the notion of a k-core — a subnetwork whose members each retain at least k connections — and the related core number of each account, the largest k for which it still belongs to the k-core. Comparing accounts, they find that this coreness lines up with real spreading power better than degree, closeness, or betweenness do. They also discover a practical shortcut: certain accounts that sit just outside the innermost core turn out to spread a clip just as far, which spares them from having to reach the hardest-to-contact celebrities.

Questions 1–10

Q1. [MCQ] How does the team define an “internet meme”?

- **A.** A formal technical object with a precise mathematical definition fixed by the platform.
- **B.** Only a humorous image, never a video or a piece of text, that is shared online.
- **C.** Anything — image, video, or text — that spreads across the internet.
- **D.** A private message sent between two accounts that no other account can ever see.

Answer: C

Reference: Lecture 159 – Pseudo Cores: Introduction (≈01:39–end)

Solution: An internet meme is informally just anything — a picture, a video, a piece of text — that spreads across the internet. It is not a precise technical term, not restricted to humorous images, and not a private one-to-one message.

Q2. [MCQ] Of the three levers, which one does the team actually act on to make the clip go viral?

A. They lower the quality of the clip, on the theory that worse content travels further.
B. They redesign the platform's follower network so that everything becomes popular.
C. They choose which accounts to seed, since the network structure is fixed for everyone.
D. They wait passively without seeding any account, letting the clip spread on its own.

Answer: C

Reference: Lecture 160 – How to be Viral (≈02:17–end)

Solution: Quality matters and structure matters, but the structure is fixed and the same for everyone, so the lever the team can pull is choosing the right accounts to seed. They cannot redesign the network, would not lower quality, and passive waiting is not a strategy.

Q3. [Match the following] Match each term in Column A with its description in Column B.

Column A (term)	Column B (description)
1. k-core	a. the largest k for which a node stays in the k-core
2. Core number	b. accounts outside the innermost core with similar spreading power
3. Core–periphery	c. a subgraph where every node has degree at least k
4. Pseudo-core	d. a dense core surrounded by a sparse periphery

A. 1-c, 2-a, 3-b, 4-d
B. 1-c, 2-d, 3-a, 4-b
C. 1-a, 2-c, 3-d, 4-b
D. 1-c, 2-a, 3-d, 4-b

Answer: D (1-c, 2-a, 3-d, 4-b)

Reference: Lectures 162, 165–166 – k-core, core number, pseudo-core

Solution: A k-core is a subgraph where every node has degree at least k (1-c); the core number is the largest such k for a node (2-a); core–periphery is a dense core with a sparse periphery (3-d); and a pseudo-core is an outside-the-core account with similar spreading power (4-b).

Q4. [MCQ] What is the definition of the k-core of a graph?

A. The single node in the graph that happens to have the highest degree of all.
B. A subgraph in which every node has a degree of exactly k and no more.
C. A subgraph in which every node has a degree strictly less than k.
D. A subgraph in which every node has a degree of at least k.

Answer: D

Reference: Lecture 162 – Finding the right key nodes (≈02:37–end)

Solution: A k-core is a (maximal) subgraph in which every node has degree at least k. It is not “exactly k,” not “less than k,” and not a single highest-degree node.

Q5. [MCQ] Omar runs the snippet below. What does coreness hold afterward?

```
import networkx as nx
coreness = nx.core_number(G)
print(coreness[account])
```

A. Each account's degree, that is, its number of direct followers in the network.
B. Each account's core number, found by k-shell decomposition of the network.
C. The single largest k-core of the whole network, returned as one list of accounts.
D. Each account's betweenness, that is, how often it lies on shortest paths.

Answer: B

Reference: Lecture 165 – Importance of core nodes in cascading (≈08:39–end)

Solution: nx.core_number returns a dictionary mapping each node to its core number, computed by k-shell decomposition. It is not the plain degree, not a single k-core list, and not betweenness.

Q6. [Statement type] Consider the two statements about coreness and decide their truth.

Statement I: Coreness predicts an account's spreading power better than degree, closeness, or betweenness.

Statement II: An account's core number is the largest k for which it belongs to the k-core.

A. Both statements are true.
B. Both statements are false.
C. Statement I is true, but Statement II is false.
D. Statement I is false, but Statement II is true.

Answer: A

Reference: Lectures 162, 165 – Core number and influence

Solution: Both statements are correct: experiments find coreness lines up with spreading power better than degree, closeness, or betweenness, and the core number is defined as the largest k for which a node is still in the k-core. So both hold.

Q7. [MCQ] What is a “pseudo-core,” and why is it useful to the team?

A. Accounts deep in the innermost core, which are the easiest of all to contact and seed.
B. Accounts outside the innermost core with similar spreading power, easier to reach.
C. Accounts in the outermost periphery, which never spread anything to anyone at all.
D. Accounts with the very lowest core number, useful because they cost nothing to seed.

Answer: B

Reference: Lecture 166 – Pseudo core (≈03:05–end)

Solution: Pseudo-cores are accounts that sit outside the innermost core yet spread a clip about as far, so the team can seed them instead of the hard-to-reach core celebrities. They are not the innermost core, not inert periphery accounts, and not chosen for being lowest-coreness.

Q8. [MCQ] Why does seeding a single core account tend to make a clip go viral?

A. The core is densely interconnected, so the clip races through it and out to the periphery.
B. The core is completely cut off from the periphery, so the clip stays trapped inside it.
C. The core consists of isolated accounts with no followers, which forces wide sharing.
D. The periphery is denser than the core, so seeding the core reaches almost no one.

Answer: A

Reference: Lecture 162 – Finding the right key nodes (≈00:45–end)

Solution: Because the core is densely interconnected, an idea seeded there spreads quickly across the whole core and then out to the well-connected periphery. The core is not cut off from the periphery, not made of isolated accounts, and the periphery is sparser than the core, not denser.

Q9. [MSQ] Which statements about cores and viral spreading are correct? (Select all that apply.)

I. A k-core is a subgraph where every node has degree at least k.
II. Coreness aligns with spreading power better than degree or betweenness.
III. Pseudo-cores can spread a clip about as far as the innermost core.
IV. The platform's network structure can be freely redesigned to force virality.

A. I, II, IV
B. II, III, IV
C. I, III
D. I, II, III

Answer: D (statements I, II, III)

Reference: Lectures 160–166 – Cores, coreness, pseudo-cores

Solution: The degree-at-least-k definition (I), coreness as the best influence predictor (II), and pseudo-cores matching the core's reach (III) are all correct. The network structure is fixed and the same for everyone, so IV is false.

Q10. [MCQ] In a hierarchical (core–periphery) network, which accounts make up the core?

A. The sparsely connected, low-influence accounts that sit at the network's outer edge.
B. The densely interconnected, high-influence accounts at the centre of the network.
C. Accounts chosen at random, since position in the network has nothing to do with the core.
D. Whichever accounts joined the platform most recently, regardless of their connections.

Answer: B

Reference: Lecture 162 – Finding the right key nodes (≈00:00–00:45)

Solution: The core is the small set of densely interconnected, high-influence accounts at the centre; the periphery is the sparse outer edge. The core is not the sparse edge, not a random set, and not defined by join date.

Comprehension 2 — Peeling the Network: k-Shell Decomposition

Case Study

The same team implements k-shell decomposition to find the core and to rank accounts by influence. The procedure peels the network in rounds. With an empty bucket labelled 1, it removes every node of degree 1; pulling a node out also strips away its edges, so a neighbour can lose enough links to fall to degree 1 as well, and those newly-exposed nodes are removed into the same bucket until none of degree 1 remain. The bucket is set aside, a new bucket labelled 2 is opened, and nodes of degree 2 or less are peeled the same way, and so on, until the network is empty.

An important subtlety is that the buckets are shells, not the cores themselves. The k-core is recovered as the union of all buckets from k upward, because every node in a higher shell trivially satisfies the lower-degree requirement. The nodes removed in the very last bucket form the innermost shell — the core. NetworkX exposes this directly through core numbers, and the team confirms that ranking accounts by core number tracks their cascade influence more closely than degree, closeness, or betweenness.

Finally, to estimate each shell's reach, the team runs an independent-cascade simulation many times from seeds drawn out of that shell and averages how many accounts end up infected — the shell's cascade capacity. Plotting cascade capacity against shell number, they see it rise and then level off well before the innermost shell, which is exactly why some outer-shell (pseudo-core) accounts spread a clip as far as the core. Key fragments:

```
import networkx as nx

# core number of every node (runs k-shell decomposition internally)
cn = nx.core_number(G)

# one peeling round into the bucket labelled k
while any(G.degree(v) <= k for v in G.nodes):
    low = [v for v in G.nodes if G.degree(v) <= k]
    bucket_k.extend(low)
    G.remove_nodes_from(low)        # deleting edges may expose new low-degree nodes
```

Questions 1–10

Q1. [MCQ] In the k-shell decomposition, what does each peeling round do?

A. Removes only the single highest-degree node and stops the round immediately.
B. Removes a fixed number of nodes chosen at random, regardless of their degree.
C. Recursively removes nodes of degree k or less until none remain, then moves to k+1.
D. Adds new nodes of degree k to the network until the graph stops growing.

Answer: C

Reference: Lecture 162 – Finding the right key nodes (≈04:39–end)

Solution: Each round recursively peels away nodes of degree k or less until none are left, then the threshold k increases by one. It is not random removal, not a single high-degree node, and it removes rather than adds nodes.

Q2. [MCQ] Why can removing one low-degree node expose more nodes to remove in the same round?

A. Removing a node has no effect on any other node's degree, so nothing new is exposed.
B. Removing a node randomly raises some neighbours' degrees, which must then be removed too.
C. Removing a node adds new edges elsewhere, lowering far-away nodes below the threshold.
D. Removing a node also deletes its edges, which can drop a neighbour's degree below the threshold.

Answer: D

Reference: Lecture 162 – Finding the right key nodes (≈04:39–end)

Solution: Deleting a node also deletes its incident edges, so a neighbour's degree can fall to or below the current threshold and must be peeled in the same bucket. Removal lowers neighbours' degrees, does not raise them, adds no edges, and certainly does affect neighbours.

Q3. [MCQ] The peeling round below fills bucket_k. With threshold k = 1, what gets placed in the bucket?

```
while any(G.degree(v) <= k for v in G.nodes):   # k = 1
    low = [v for v in G.nodes if G.degree(v) <= k]
    bucket_k.extend(low); G.remove_nodes_from(low)
```

A. Every node of degree 1, including any new degree-1 nodes exposed as edges are deleted.
B. Only the nodes that had degree 1 at the very start, ignoring any exposed later.
C. Every node of degree 2 or more, leaving the degree-1 nodes untouched in the graph.
D. A single degree-1 node, after which the while-loop stops without removing more.

Answer: A

Reference: Lecture 162 – Finding the right key nodes (≈04:39–end)

Solution: The loop keeps removing while any node has degree ≤ 1, so it peels every degree-1 node — including ones newly exposed when edges vanish — into bucket_k. It does not stop at the initial set, does not spare degree-1 nodes, and does not halt after a single removal.

Q4. [Arrange in order] Arrange the steps of k-shell decomposition in the correct order.

Steps (in scrambled order):

(i) Set the threshold to k = 1 and open an empty bucket for it.
(ii) Repeatedly remove every node of degree ≤ k into the current bucket.
(iii) When no node of degree ≤ k remains, set the bucket aside.
(iv) Increase k by one and open a new bucket.
(v) Stop when the network is empty; the last bucket is the innermost shell (core).

A. i → ii → iii → iv → v
B. ii → i → iii → iv → v
C. i → iii → ii → iv → v
D. i → ii → iv → iii → v

Answer: **A (i → ii → iii → iv → v)**

Reference: Lecture 162 – Finding the right key nodes (k-shell)

Solution: Begin at k = 1 with an empty bucket (i), peel all nodes of degree ≤ k into it (ii), set the bucket aside when none remain (iii), raise k and open a new bucket (iv), and stop when the graph is empty, the last bucket being the core (v).

Q5. [MCQ] After the decomposition, which nodes form the core of the network?

A. The nodes peeled in the very first bucket, the outermost shell of the network.
B. A random selection of nodes taken evenly from across all of the buckets.
C. The nodes peeled in the very last bucket, the innermost shell of the network.
D. Every node that was ever placed in any bucket during the whole procedure.

Answer: **C**

Reference: Lecture 162 – Finding the right key nodes (≈09:09–end)

Solution: The core is the innermost shell — the nodes removed in the last bucket, which survived every earlier peeling round. The first bucket is the outermost shell, and the core is neither a random sample nor all bucketed nodes.

Q6. [Statement type] Consider the two statements about k-shell decomposition and the k-core, and decide their truth.

Statement I: Peeling a low-degree node can drop a neighbour below the threshold, exposing it for removal.
Statement II: The k-core is exactly the single bucket k, excluding all the higher-numbered buckets.

A. Both statements are true.
B. Both statements are false.
C. Statement I is true, but Statement II is false.
D. Statement I is false, but Statement II is true.

Answer: **C**

Reference: Lecture 162 – Finding the right key nodes (k-shell)

Solution: The first statement is correct: deleting a node's edges can push a neighbour below the threshold. The second is wrong: the k-core is the union of bucket k with all higher buckets, since higher-shell nodes trivially meet the lower-degree requirement. So the first holds and the second fails.

Q7. [MCQ] In the team's influence experiment, which centrality best matched an account's cascade power?

A. Degree, the raw number of followers, which always equalled the spreading power exactly.
B. Coreness (core number), which tracked cascade influence more closely than the others.
C. Closeness alone, which matched perfectly while coreness matched not at all.
D. Betweenness alone, which a node with low coreness always exceeds in spreading power.

Answer: **B**

Reference: Lecture 165 – Importance of core nodes in cascading (≈19:35–end)

Solution: The experiment found coreness tracked cascade influence most closely — more than degree, closeness, or betweenness. A high-degree node can have weak influence, closeness alone is not a perfect match, and betweenness is not the deciding factor.

Q8. [MCQ] How is a shell's "cascade capacity" estimated in the simulation?

A. By seeding from that shell, running the cascade many times, and averaging how many get infected.
B. By counting the number of nodes in the shell, ignoring how far any cascade actually spreads.
C. By running the cascade exactly once from the whole network and reading off a single number.
D. By measuring the shortest path from the shell to the innermost core of the network.

Answer: A

Reference: Lecture 166 – Pseudo core (≈03:05–end)

Solution: Cascade capacity is found by seeding the independent-cascade model from that shell, running it many times, and averaging the final number infected. It is not the shell's size, not a single run from the whole graph, and not a shortest-path measure.

Q9. [MCQ] Plotting cascade capacity against shell number, what shape reveals the pseudo-cores?

A. It rises steadily right up to the innermost shell, so only the core ever spreads widely.
B. It rises and then levels off before the innermost shell, so some outer shells match the core.
C. It falls steadily with shell number, so the outermost shell has the greatest reach of all.
D. It stays flat across every shell, so the choice of seeding shell makes no difference at all.

Answer: B

Reference: Lecture 166 – Pseudo core (≈03:46–end)

Solution: Cascade capacity rises and then plateaus before reaching the innermost shell, so accounts in those plateaued outer shells — the pseudo-cores — spread about as far as the core. It does not climb only at the core, fall with shell number, or stay flat.

Q10. [MSQ] Which statements about the decomposition and cascades are correct? (Select all that apply.)

I. Each round peels nodes of degree ≤ k until none remain.
II. The k-core is the union of bucket k with all higher buckets.
III. The innermost shell (last bucket) is the core of the network.
IV. Every shell has exactly the same cascade capacity as every other shell.

A. I, II, IV
B. II, III, IV
C. I, III
D. I, II, III

Answer: D (statements I, II, III)

Reference: Lectures 162, 165–166 – k-shell, core, cascade capacity

Solution: Peeling degree-≤-k nodes each round (I), the k-core as a union of bucket k and higher (II), and the last bucket being the core (III) are all correct. Cascade capacity varies across shells — rising then plateauing — so IV is false.